

Chiến lược thiết kế thuật toán: Chia để trị

Vu Tien Dat

1 Giới thiệu chung

Chia để trị (Divide and Conquer) là một trong những chiến lược quan trọng nhất trong thiết kế thuật toán. Tư tưởng chính:

“Chia bài toán lớn thành các bài toán con nhỏ hơn, giải chúng một cách độc lập (thường bằng đệ quy), sau đó tổng hợp kết quả để được lời giải cho bài toán ban đầu.”

2 Ba bước cơ bản

Mọi thuật toán chia để trị đều tuân theo ba bước:

1. **Chia (Divide)**: Phân bài toán kích thước n thành a bài toán con, mỗi bài có kích thước $\frac{n}{b}$ (thường $a = b$, ví dụ: 2 bài con kích thước $n/2$).
2. **Trị (Conquer)**: Giải đệ quy các bài toán con. Nếu bài toán con đủ nhỏ (kích thước cơ sở), giải trực tiếp.
3. **Tổng hợp (Combine)**: Kết hợp các lời giải của bài toán con để tạo lời giải cho bài toán gốc.

3 Lược đồ giải thuật tổng quát

Giả sử hàm $\text{DIVIDEANDCONQUER}(P)$ giải bài toán P có kích thước n .

```
0: function DIVIDEANDCONQUER( $P$ )
0:   if  $n \leq n_0$  then {Trường hợp cơ sở}
0:     return SOLVEDIRECTLY( $P$ )
0:   else
0:     Chia  $P$  thành  $P_1, P_2, \dots, P_a$  {Bước chia}
0:     for  $i = 1$  to  $a$  do
0:        $s_i \leftarrow \text{DIVIDEANDCONQUER}(P_i)$  {Bước trị (đệ quy)}
0:     end for
0:      $s \leftarrow \text{COMBINE}(s_1, s_2, \dots, s_a)$  {Bước tổng hợp}
0:     return  $s$ 
0:   end if
0: end function=0
```

Trong đó:

- n_0 : ngưỡng đủ nhỏ để giải trực tiếp (thường $n_0 = 1$ hoặc $n_0 = 2$).
- Hàm SOLVEDIRECTLY : giải bài toán con rất nhỏ.
- Hàm COMBINE : tổng hợp kết quả.

4 Hệ thức truy hồi tổng quát

Gọi $T(n)$ là thời gian thực hiện thuật toán chia để trị với kích thước đầu vào n . Ta có:

$$T(n) = \begin{cases} O(1) & \text{nếu } n \leq n_0 \\ a \cdot T\left(\frac{n}{b}\right) + f(n) & \text{nếu } n > n_0 \end{cases}$$

Trong đó:

- a : số lượng bài toán con (≥ 1).
- b : hệ số giảm kích thước ($b > 1$).
- $f(n)$: chi phí để chia bài toán và tổng hợp kết quả.

5 Định lý thặng (Master Theorem) để đánh giá

Cho $T(n) = aT\left(\frac{n}{b}\right) + f(n)$ với $a \geq 1, b > 1$.

So sánh $f(n)$ với $n^{\log_b a}$:

1. Nếu $f(n) = O(n^{\log_b a - \varepsilon})$ với $\varepsilon > 0$:

$$T(n) = \Theta(n^{\log_b a})$$

2. Nếu $f(n) = \Theta(n^{\log_b a} \log^k n)$ với $k \geq 0$:

$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$$

3. Nếu $f(n) = \Omega(n^{\log_b a + \varepsilon})$ và $af\left(\frac{n}{b}\right) \leq cf(n)$ với $c < 1$:

$$T(n) = \Theta(f(n))$$

6 Ví dụ minh họa: Tìm max và min trong mảng

Xét bài toán tìm cả phần tử lớn nhất và nhỏ nhất của mảng $A[1..n]$. Thuật toán chia để trị:

```

0: function MINMAX( $A, l, r$ )
0:   if  $l = r$  then {1 phần tử}
0:     return ( $A[l], A[l]$ )
0:   else if  $l + 1 = r$  then {2 phần tử}
0:     return ( $\min(A[l], A[r]), \max(A[l], A[r])$ )
0:   else
0:      $m \leftarrow \lfloor (l + r) / 2 \rfloor$ 
0:     ( $\min1, \max1$ )  $\leftarrow$  MINMAX( $A, l, m$ )
0:     ( $\min2, \max2$ )  $\leftarrow$  MINMAX( $A, m + 1, r$ )
0:      $\min\_all \leftarrow \min(\min1, \min2)$ 
0:      $\max\_all \leftarrow \max(\max1, \max2)$ 
0:     return ( $\min\_all, \max\_all$ )
0:   end if
0: end function

```

6.1 Phân tích độ phức tạp

Số phép so sánh $T(n)$ thỏa mãn:

$$T(n) = \begin{cases} 0 & n = 1 \\ 1 & n = 2 \\ 2T\left(\frac{n}{2}\right) + 2 & n > 2 \end{cases}$$

Giải hệ thức trên với $n = 2^k$:

$$\begin{aligned}
T(n) &= 2T\left(\frac{n}{2}\right) + 2 \\
&= 2\left[2T\left(\frac{n}{4}\right) + 2\right] + 2 = 4T\left(\frac{n}{4}\right) + 4 + 2 \\
&= \dots \\
&= 2^{k-1}T(2) + \sum_{i=1}^{k-1} 2^i \\
&= 2^{k-1} \cdot 1 + (2^k - 2) \\
&= \frac{3n}{2} - 2
\end{aligned}$$

Vậy $T(n) = O(n)$. Cách này tốt hơn phương pháp duyệt tuần tự cần $2n - 2$ phép so sánh.

7 Các bài toán kinh điển áp dụng chia để trị

Bài toán	Công thức truy hồi	Độ phức tạp	Ghi chú
Tìm kiếm nhị phân	$T(n) = T(n/2) + O(1)$	$O(\log n)$	
Sắp xếp trộn (Merge Sort)	$T(n) = 2T(n/2) + O(n)$	$O(n \log n)$	
Sắp xếp nhanh (QuickSort)	$T(n) = T(k) + T(n - k - 1) + O(n)$	TB: $O(n \log n)$	Xấu: $O(n^2)$
Nhân Strassen	$T(n) = 7T(n/2) + O(n^2)$	$O(n^{\log_2 7}) \approx O(n^{2.81})$	Nhân ma trận
Tìm cặp điểm gần nhất	$T(n) = 2T(n/2) + O(n)$	$O(n \log n)$	Hình học

8 Lưu ý khi học cho sinh viên

- **Đệ quy là trái tim** của chia để trị — cần thành thạo viết và phân tích đệ quy.
- **Trường hợp cơ sở** phải đúng, nếu không thuật toán sẽ không dừng.
- **Bước tổng hợp** quyết định độ phức tạp: nếu tổng hợp tốn $O(n^2)$ thì dù chia tốt cũng không hiệu quả.
- Không phải lúc nào chia để trị cũng tối ưu: với dữ liệu nhỏ, thuật toán đơn giản có thể nhanh hơn.

9 Ví dụ 1

Xét một thuật toán A giải quyết bài toán bằng cách:

- Chia bài toán ban đầu thành 5 bài toán con, mỗi bài toán có kích thước bằng một nửa bài toán gốc.
- Sau đó giải đệ quy từng bài toán con.
- Cuối cùng kết hợp các lời giải lại với nhau trong thời gian tuyến tính (linear time).

Hỏi: Độ phức tạp của thuật toán này là gì?

9.1 Thiết lập hệ thức truy hồi

Gọi $T(n)$ là thời gian thực hiện thuật toán với kích thước đầu vào n .

Theo mô tả:

- Chia bài toán thành $a = 5$ bài toán con.
- Mỗi bài toán con có kích thước $\frac{n}{2} \Rightarrow b = 2$.
- Thời gian kết hợp là tuyến tính: $f(n) = O(n)$.

Vậy ta có hệ thức truy hồi:

$$T(n) = 5 \cdot T\left(\frac{n}{2}\right) + O(n)$$

9.2 Áp dụng Định lý Thợ (Master Theorem)

Định lý Thợ cho hệ thức:

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

với $a \geq 1, b > 1$.

9.2.1 Bước 1: Tính $n^{\log_b a}$

$$\begin{aligned}\log_b a &= \log_2 5 \\ n^{\log_b a} &= n^{\log_2 5}\end{aligned}$$

Ta biết:

$$\log_2 4 = 2, \quad \log_2 8 = 3$$

Vì $4 < 5 < 8$ nên:

$$2 < \log_2 5 < 3$$

Cụ thể:

$$\log_2 5 \approx 2.321928$$

9.2.2 Bước 2: So sánh $f(n)$ với $n^{\log_b a}$

$$\begin{aligned}f(n) &= O(n) \quad (\text{tuyến tính}) \\ n^{\log_b a} &= n^{\log_2 5} \approx n^{2.3219}\end{aligned}$$

Rõ ràng:

$$n = n^1 \quad \text{và} \quad 1 < \log_2 5 \approx 2.3219$$

Do đó:

$$f(n) = O(n^{\log_b a - \varepsilon}) \quad \text{với} \quad \varepsilon = \log_2 5 - 1 > 0$$

9.2.3 Bước 3: Kết luận theo trường hợp 1 của Master Theorem

Trường hợp 1: Nếu $f(n) = O(n^{\log_b a - \varepsilon})$ với $\varepsilon > 0$ thì:

$$T(n) = \Theta(n^{\log_b a})$$

Vậy:

$$T(n) = \Theta(n^{\log_2 5})$$

9.3 Kết quả cuối cùng

$$T(n) = \Theta(n^{\log_2 5}) \approx \Theta(n^{2.321928})$$

9.4 Nhận xét

- Thuật toán này có độ phức tạp **lớn hơn** $O(n^2)$ nhưng **nhỏ hơn** $O(n^3)$.
- So sánh với một số thuật toán nổi tiếng:
 - Sắp xếp trộn (Merge Sort): $O(n \log n)$
 - Nhân ma trận thông thường: $O(n^3)$
 - Nhân ma trận Strassen: $O(n^{\log_2 7}) \approx O(n^{2.807})$
 - Thuật toán này: $O(n^{\log_2 5}) \approx O(n^{2.322})$
- Thuật toán A chậm hơn Strassen một chút nhưng nhanh hơn nhiều so với giải pháp bậc ba $O(n^3)$.

10 Ví dụ 2

Tương tự như Ví dụ 1, một thuật toán B giải bài toán kích thước n bằng cách:

- Đệ quy giải 2 bài toán con, mỗi bài toán có kích thước $n - 1$.
- Sau đó kết hợp các lời giải trong thời gian hằng số (constant time).

Hỏi: Độ phức tạp của thuật toán này là gì?

10.1 Thiết lập hệ thức truy hồi

Gọi $T(n)$ là thời gian thực hiện thuật toán B với kích thước đầu vào n .

Theo mô tả:

- Chia bài toán thành $a = 2$ bài toán con.
- Mỗi bài toán con có kích thước $n - 1$ (giảm đi 1 đơn vị, **không phải** chia theo tỷ lệ).
- Thời gian kết hợp là hằng số: $f(n) = O(1)$.

Vậy ta có hệ thức truy hồi:

$$T(n) = 2 \cdot T(n - 1) + O(1)$$

10.2 Điểm đặc biệt: Không áp dụng được Master Theorem

Lưu ý quan trọng: Định lý Thợ (Master Theorem) chỉ áp dụng cho hệ thức dạng:

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

với $b > 1$ (kích thước bài toán con giảm theo tỷ lệ).

Ở đây, kích thước bài toán con là $n - 1$ (giảm tuyến tính, không phải giảm theo tỷ lệ). Do đó, ta **không thể dùng Master Theorem** mà phải giải trực tiếp bằng phương pháp khai triển hoặc phương pháp thế.

10.3 Giải hệ thức truy hồi

10.3.1 Bước 1: Khai triển lặp

Ta có:

$$T(n) = 2T(n - 1) + c$$

với c là hằng số (thời gian kết hợp).

Tiếp tục khai triển $T(n - 1)$:

$$T(n - 1) = 2T(n - 2) + c$$

$$T(n) = 2[2T(n - 2) + c] + c = 4T(n - 2) + 2c + c = 4T(n - 2) + (2^2 - 1)c$$

Khai triển $T(n - 2)$:

$$T(n - 2) = 2T(n - 3) + c$$

$$T(n) = 4[2T(n - 3) + c] + 3c = 8T(n - 3) + 4c + 3c = 8T(n - 3) + (2^3 - 1)c$$

10.3.2 Bước 2: Tìm quy luật

Sau k bước khai triển:

$$T(n) = 2^k T(n - k) + (2^k - 1)c$$

10.3.3 Bước 3: Điều kiện dừng

Khi $n - k = 1$ (bài toán cơ sở), ta có $k = n - 1$. Gọi $T(1) = c_0$ là hằng số (trường hợp cơ sở).

Thay vào:

$$T(n) = 2^{n-1}T(1) + (2^{n-1} - 1)c$$

$$T(n) = 2^{n-1}c_0 + c \cdot 2^{n-1} - c$$

$$T(n) = (c_0 + c) \cdot 2^{n-1} - c$$

10.3.4 Bước 4: Kết luận

Vì $(c_0 + c)$ và c là các hằng số:

$$T(n) = O(2^{n-1}) = O(2^n)$$

10.4 Kết quả cuối cùng

$$T(n) = O(2^n)$$

Độ phức tạp của thuật toán B là **hàm mũ**.

10.5 Nhận xét và so sánh

- Thuật toán B có độ phức tạp **rất lớn** ($O(2^n)$), không khả thi cho n lớn.
- Nguyên nhân: mỗi bước chỉ giảm kích thước đi 1 nhưng lại nhân đôi số lượng bài toán con.
- So sánh với Ví dụ 1:
 - Ví dụ 1: $T(n) = 5T(n/2) + O(n) \Rightarrow \Theta(n^{\log_2 5}) \approx \Theta(n^{2.32})$ (đa thức).
 - Ví dụ 2: $T(n) = 2T(n-1) + O(1) \Rightarrow O(2^n)$ (hàm mũ).
- Đây là ví dụ điển hình cho thấy việc **giảm kích thước theo hằng số** (như $n-1$) thường dẫn đến độ phức tạp mũ, trong khi **giảm theo tỷ lệ** (như $n/2$) dẫn đến độ phức tạp đa thức hoặc logarit.

11 Ví dụ 3

Lại tương tự như Ví dụ 1, một thuật toán C giải bài toán kích thước n bằng cách:

- Chia bài toán thành 9 bài toán con, mỗi bài toán có kích thước $n/3$.
- Sau đó giải đệ quy từng bài toán con.
- Cuối cùng kết hợp các lời giải trong thời gian $O(n^2)$.

Hỏi: Độ phức tạp của thuật toán này là gì?

11.1 Thiết lập hệ thức truy hồi

Gọi $T(n)$ là thời gian thực hiện thuật toán C với kích thước đầu vào n .

Theo mô tả:

- Số bài toán con: $a = 9$
- Mỗi bài toán con có kích thước: $\frac{n}{3} \Rightarrow b = 3$
- Thời gian kết hợp: $f(n) = O(n^2)$

Vậy ta có hệ thức truy hồi:

$$T(n) = 9 \cdot T\left(\frac{n}{3}\right) + O(n^2)$$

11.2 Áp dụng Định lý Thợ (Master Theorem)

Định lý Thợ cho hệ thức:

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

với $a \geq 1, b > 1$.

11.2.1 Bước 1: Tính $n^{\log_b a}$

$$\log_b a = \log_3 9$$

Vì $9 = 3^2$, ta có:

$$\log_3 9 = 2$$

Do đó:

$$n^{\log_b a} = n^2$$

11.2.2 Bước 2: So sánh $f(n)$ với $n^{\log_b a}$

$$f(n) = O(n^2)$$
$$n^{\log_b a} = n^2$$

Rõ ràng:

$$f(n) = \Theta(n^2) = \Theta(n^{\log_b a})$$

11.2.3 Bước 3: Xác định trường hợp của Master Theorem

Đây là **Trường hợp 2** của Master Theorem:

$$f(n) = \Theta(n^{\log_b a} \cdot \log^k n) \quad \text{với } k = 0$$

Theo định lý:

$$T(n) = \Theta(n^{\log_b a} \cdot \log^{k+1} n) = \Theta(n^2 \cdot \log^1 n)$$

11.2.4 Bước 4: Kết luận

$$T(n) = \Theta(n^2 \log n)$$

11.3 Kết quả cuối cùng

$$T(n) = \Theta(n^2 \log n)$$

11.4 Nhận xét và so sánh

- Thuật toán C có độ phức tạp $O(n^2 \log n)$, nhanh hơn so với giải pháp ngây thơ $O(n^3)$ hoặc $O(n^4)$.
- So sánh với các thuật toán khác:
 - Ví dụ 1: $T(n) = 5T(n/2) + O(n) \Rightarrow \Theta(n^{\log_2 5}) \approx \Theta(n^{2.322})$ (đa thức, không có log)
 - Ví dụ 2: $T(n) = 2T(n-1) + O(1) \Rightarrow O(2^n)$ (hàm mũ, rất tệ)
 - Ví dụ 3: $T(n) = 9T(n/3) + O(n^2) \Rightarrow \Theta(n^2 \log n)$ (đa thức nhân log)
- Lý do ra $\Theta(n^2 \log n)$: vì thời gian kết hợp $O(n^2)$ cân bằng chính xác với $n^{\log_3 9} = n^2$, tạo ra thêm hệ số $\log n$.

11.5 Bảng tóm tắt ba ví dụ

Ví dụ	Hệ thức	Trường hợp MT	Độ phức tạp
Ví dụ 1	$T(n) = 5T(n/2) + O(n)$	TH1	$\Theta(n^{\log_2 5}) \approx \Theta(n^{2.322})$
Ví dụ 2	$T(n) = 2T(n-1) + O(1)$	Không áp dụng	$O(2^n)$
Ví dụ 3	$T(n) = 9T(n/3) + O(n^2)$	TH2	$\Theta(n^2 \log n)$

11.6 Bài tập tự luyện thêm

Hãy tự giải các trường hợp sau bằng Master Theorem:

- $T(n) = 4T(n/2) + O(n^2)$
- $T(n) = 8T(n/2) + O(n^3)$
- $T(n) = 7T(n/2) + O(n^2)$
- $T(n) = 2T(n/2) + O(n \log n)$

Đáp án gợi ý:

- $\Theta(n^2 \log n)$

2. $\Theta(n^3 \log n)$
3. $\Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$
4. $\Theta(n \log^2 n)$

12 Ví dụ 4

Cho đoạn mã giả sau:

```
public void function(int n) {
    if (n > 1) {
        System.out.println("*");
        function(n/2);
        function(n/2);
    }
}
```

Yêu cầu: Viết hệ thức truy hồi và giải tìm độ phức tạp thuật toán.

12.1 Phân tích đoạn mã

- Hàm `function(n)` được gọi với tham số n .
- **Điều kiện dừng:** Khi $n \leq 1$, hàm không làm gì (trường hợp cơ sở).
- **Khi $n > 1$:**
 1. In ra một dấu “*”: mất $O(1)$ thời gian.
 2. Gọi đệ quy `function(n/2)` **hai lần**.

12.2 Thiết lập hệ thức truy hồi

Gọi $T(n)$ là số bước (hoặc thời gian) thực hiện khi gọi `function(n)`.

Với $n > 1$:

$$T(n) = 2 \cdot T\left(\frac{n}{2}\right) + O(1)$$

Với $n \leq 1$ (trường hợp cơ sở):

$$T(1) = O(1)$$

Viết gọn lại:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(1)$$

12.3 Giải hệ thức truy hồi

12.3.1 Cách 1: Áp dụng Master Theorem

Hệ thức có dạng:

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

với:

$$a = 2, \quad b = 2, \quad f(n) = O(1)$$

Tính:

$$\log_b a = \log_2 2 = 1$$

$$n^{\log_b a} = n^1 = n$$

So sánh $f(n) = O(1)$ với $n^{\log_b a} = n$:

$$1 < n \Rightarrow f(n) = O(n^{\log_b a - \varepsilon}) \text{ với } \varepsilon = 1$$

Đây là **Trường hợp 1** của Master Theorem:

$$T(n) = \Theta(n^{\log_b a}) = \Theta(n)$$

12.3.2 Cách 2: Khai triển (để kiểm tra)

Giả sử $n = 2^k$ và $T(1) = c_0$, $f(n) = c$ (hằng số).

Khai triển:

$$\begin{aligned}T(n) &= 2T\left(\frac{n}{2}\right) + c \\&= 2\left[2T\left(\frac{n}{4}\right) + c\right] + c = 4T\left(\frac{n}{4}\right) + 2c + c \\&= 4\left[2T\left(\frac{n}{8}\right) + c\right] + 3c = 8T\left(\frac{n}{8}\right) + 4c + 3c \\&= \dots \\&= 2^k T(1) + (2^{k-1} + 2^{k-2} + \dots + 2^0)c\end{aligned}$$

Vì $2^{k-1} + 2^{k-2} + \dots + 2^0 = 2^k - 1$, ta có:

$$T(n) = 2^k \cdot c_0 + (2^k - 1)c$$

Mà $n = 2^k \Rightarrow k = \log_2 n$, suy ra:

$$T(n) = n \cdot c_0 + (n - 1)c = O(n)$$

12.4 Kết quả cuối cùng

$$T(n) = \Theta(n)$$

12.5 Giải thích ý nghĩa

- Mặc dù có hai lời gọi đệ quy, nhưng mỗi lần kích thước bài toán giảm một nửa, nên tổng số công việc chỉ là tuyến tính.
- Cụ thể: số lần in “*” đúng bằng số nút trong cây đệ quy. Với cây nhị phân đầy đủ có chiều cao $\log_2 n$, số nút là $2^{\log_2 n + 1} - 1 \approx 2n - 1 = O(n)$.
- Đây là một ví dụ điển hình: dù có hai nhánh đệ quy, nhưng vì độ sâu chỉ $\log n$ nên tổng số lần gọi hàm là $O(n)$, không phải $O(2^{\log n}) = O(n)$.

12.6 Bảng tóm tắt các ví dụ đã làm

Ví dụ	Hệ thức	Trường hợp MT	Độ phức tạp
Ví dụ 1	$T(n) = 5T(n/2) + O(n)$	TH1	$\Theta(n^{\log_2 5})$
Ví dụ 2	$T(n) = 2T(n-1) + O(1)$	Không áp dụng	$O(2^n)$
Ví dụ 3	$T(n) = 9T(n/3) + O(n^2)$	TH2	$\Theta(n^2 \log n)$
Ví dụ 4	$T(n) = 2T(n/2) + O(1)$	TH1	$\Theta(n)$

12.7 Bài tập tự luyện

Viết hệ thức truy hồi và giải cho các đoạn mã sau:

- ```
1. public void funcA(int n) {
 if (n > 1) {
 funcA(n/2);
 funcA(n/2);
 funcA(n/2);
 }
}
```
- ```
2. public void funcB(int n) {  
    if (n > 1) {  
        for (int i = 0; i < n; i++)  
            System.out.print(" ");  
    }  
}
```

```

        funcB(n/2);
    }
}

3.    public void funcC(int n) {
        if (n > 1) {
            funcC(n-1);
            funcC(n-1);
        }
    }

```

Đáp án gợi ý:

1. $T(n) = 3T(n/2) + O(1) \Rightarrow \Theta(n^{\log_2 3})$
2. $T(n) = T(n/2) + O(n) \Rightarrow \Theta(n)$ (áp dụng Master Theorem TH3 hoặc tổng hình học)
3. $T(n) = 2T(n-1) + O(1) \Rightarrow O(2^n)$ (giống Ví dụ 2)

13 Ví dụ 5

Sử dụng Định lý Thợ (Master Theorem) để giải các hệ thức truy hồi sau:

- (a) $T(n) = 3T\left(\frac{n}{2}\right) + n^2$
- (b) $T(n) = 4T\left(\frac{n}{2}\right) + n^2$
- (c) $T(n) = 16T\left(\frac{n}{4}\right) + n$
- (d) $T(n) = 2T\left(\frac{n}{2}\right) + \frac{n}{\log n}$
- (e) $T(n) = \sqrt{2}T\left(\frac{n}{2}\right) + \log n$
- (f) $T(n) = 3T\left(\frac{n}{3}\right) + \sqrt{n}$

13.1 Nhắc lại Định lý Thợ (Master Theorem)

Cho hệ thức:

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

với $a \geq 1$, $b > 1$. Đặt $c = \log_b a$.

1. **Trường hợp 1:** Nếu $f(n) = O(n^{c-\varepsilon})$ với $\varepsilon > 0$ thì $T(n) = \Theta(n^c)$.
2. **Trường hợp 2:** Nếu $f(n) = \Theta(n^c \log^k n)$ với $k \geq 0$ thì $T(n) = \Theta(n^c \log^{k+1} n)$.
3. **Trường hợp 3:** Nếu $f(n) = \Omega(n^{c+\varepsilon})$ với $\varepsilon > 0$ và $af\left(\frac{n}{b}\right) \leq df(n)$ với $d < 1$ (điều kiện chính quy) thì $T(n) = \Theta(f(n))$.

13.2 Lời giải chi tiết

13.2.1 Câu (a): $T(n) = 3T\left(\frac{n}{2}\right) + n^2$

- $a = 3, b = 2, f(n) = n^2$
- $\log_b a = \log_2 3 \approx 1.585$
- $n^{\log_b a} = n^{\log_2 3} \approx n^{1.585}$
- So sánh $f(n) = n^2$ với $n^{\log_2 3}$:

$$2 > \log_2 3 \Rightarrow f(n) = \Omega(n^{\log_2 3 + \varepsilon}) \text{ với } \varepsilon = 2 - \log_2 3 > 0$$

- Kiểm tra điều kiện chính quy:

$$af\left(\frac{n}{b}\right) = 3 \cdot \left(\frac{n}{2}\right)^2 = 3 \cdot \frac{n^2}{4} = \frac{3}{4}n^2$$

$$d = \frac{3}{4} < 1 \quad (\text{thỏa mãn})$$

- Đây là **Trường hợp 3**: $T(n) = \Theta(f(n)) = \Theta(n^2)$

$T(n) = \Theta(n^2)$

13.2.2 Câu (b): $T(n) = 4T\left(\frac{n}{2}\right) + n^2$

- $a = 4, b = 2, f(n) = n^2$
- $\log_b a = \log_2 4 = 2$
- $n^{\log_b a} = n^2$
- So sánh $f(n) = n^2$ với n^2 :

$$f(n) = \Theta(n^2) = \Theta(n^{\log_b a})$$

- Đây là **Trường hợp 2** với $k = 0$:

$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n) = \Theta(n^2 \log n)$$

$T(n) = \Theta(n^2 \log n)$

13.2.3 Câu (c): $T(n) = 16T\left(\frac{n}{4}\right) + n$

- $a = 16, b = 4, f(n) = n$
- $\log_b a = \log_4 16 = 2$ (vì $4^2 = 16$)
- $n^{\log_b a} = n^2$
- So sánh $f(n) = n$ với n^2 :

$$1 < 2 \Rightarrow f(n) = O(n^{2-\varepsilon}) \text{ với } \varepsilon = 1$$

- Đây là **Trường hợp 1**: $T(n) = \Theta(n^{\log_b a}) = \Theta(n^2)$

$T(n) = \Theta(n^2)$

13.2.4 Câu (d): $T(n) = 2T\left(\frac{n}{2}\right) + \frac{n}{\log n}$

- $a = 2, b = 2, f(n) = \frac{n}{\log n}$
- $\log_b a = \log_2 2 = 1$
- $n^{\log_b a} = n^1 = n$
- So sánh $f(n) = \frac{n}{\log n}$ với n :

$$\frac{n}{\log n} = n \cdot (\log n)^{-1}$$

- Ta có: $f(n) = \Theta(n \cdot \log^{-1} n)$. Đây là dạng $\Theta(n^c \log^k n)$ với $c = 1$ và $k = -1$.
- Tuy nhiên, Master Theorem chuẩn yêu cầu $k \geq 0$. Với $k < 0$, ta cần xét cẩn thận.
- Thực tế, $\frac{n}{\log n} = o(n)$ nên thuộc **Trường hợp 1**? Không hẳn, vì $f(n)$ nhỏ hơn n nhưng không phải là $O(n^{1-\varepsilon})$ với $\varepsilon > 0$ cố định (do $\log n$ tăng chậm).
- Với dạng này, kết quả là $T(n) = \Theta(n \log \log n)$ (có thể chứng minh bằng phương pháp đổi biến hoặc tổng).

Kết luận:

$$T(n) = \Theta(n \log \log n)$$

13.2.5 Câu (e): $T(n) = \sqrt{2}T\left(\frac{n}{2}\right) + \log n$

- $a = \sqrt{2}, b = 2, f(n) = \log n$
- $\log_b a = \log_2(\sqrt{2}) = \log_2(2^{1/2}) = \frac{1}{2}$
- $n^{\log_b a} = n^{1/2} = \sqrt{n}$
- So sánh $f(n) = \log n$ với $\sqrt{n}$:

$$\log n = o(\sqrt{n}) \Rightarrow f(n) = O\left(n^{1/2-\varepsilon}\right) \text{ với } \varepsilon = \frac{1}{2}$$

- Đây là **Trường hợp 1**: $T(n) = \Theta(n^{\log_b a}) = \Theta(\sqrt{n})$

$$T(n) = \Theta(\sqrt{n})$$

13.2.6 Câu (f): $T(n) = 3T\left(\frac{n}{3}\right) + \sqrt{n}$

- $a = 3, b = 3, f(n) = \sqrt{n} = n^{1/2}$
- $\log_b a = \log_3 3 = 1$
- $n^{\log_b a} = n^1 = n$
- So sánh $f(n) = n^{1/2}$ với n :

$$\frac{1}{2} < 1 \Rightarrow f(n) = O\left(n^{1-\varepsilon}\right) \text{ với } \varepsilon = \frac{1}{2}$$

- Đây là **Trường hợp 1**: $T(n) = \Theta(n^{\log_b a}) = \Theta(n)$

$$T(n) = \Theta(n)$$

13.3 Bảng tổng kết kết quả

Câu	Hệ thức	$\log_b a$	Trường hợp	Kết quả
(a)	$T(n) = 3T(n/2) + n^2$	≈ 1.585	TH3	$\Theta(n^2)$
(b)	$T(n) = 4T(n/2) + n^2$	2	TH2	$\Theta(n^2 \log n)$
(c)	$T(n) = 16T(n/4) + n$	2	TH1	$\Theta(n^2)$
(d)	$T(n) = 2T(n/2) + n/\log n$	1	Đặc biệt	$\Theta(n \log \log n)$
(e)	$T(n) = \sqrt{2}T(n/2) + \log n$	0.5	TH1	$\Theta(\sqrt{n})$
(f)	$T(n) = 3T(n/3) + \sqrt{n}$	1	TH1	$\Theta(n)$

13.4 Chú thích thêm cho câu (d)

Đối với câu (d): $T(n) = 2T(n/2) + \frac{n}{\log n}$, ta có thể giải chi tiết như sau:

Đặt $n = 2^k$, $T(2^k) = S(k)$. Khi đó:

$$S(k) = 2S(k-1) + \frac{2^k}{k}$$

Chia hai vế cho 2^k :

$$\frac{S(k)}{2^k} = \frac{S(k-1)}{2^{k-1}} + \frac{1}{k}$$

Đặt $U(k) = \frac{S(k)}{2^k}$, ta có:

$$U(k) = U(k-1) + \frac{1}{k}$$

Suy ra:

$$U(k) = U(0) + \sum_{i=1}^k \frac{1}{i} = \Theta(\log k)$$

Do đó:

$$S(k) = 2^k \cdot \Theta(\log k) \Rightarrow T(n) = n \cdot \Theta(\log \log n) = \Theta(n \log \log n)$$

14 Bài tập 1

Xác định độ phức tạp của các công thức sau:

(a) $T(n) = 2T(\sqrt{n}) + \log n$

(b) $T(n) = T(\sqrt{n}) + 1$

(Gợi ý: Áp dụng ý a)

14.1 Phương pháp giải dạng $T(n) = aT(n^{1/b}) + f(n)$

Khi gặp dạng truy hồi với kích thước bài toán con là $\sqrt{n}$ (hay $n^{1/b}$), ta sử dụng **phép đổi biến**:

Đặt:

$$n = 2^k \quad (\text{hoặc } n = m^k)$$

Khi đó:

$$\sqrt{n} = 2^{k/2}$$

Sau đó đặt $S(k) = T(2^k)$, ta chuyển về hệ thức truy hồi tuyến tính quen thuộc.

14.2 Câu (a): $T(n) = 2T(\sqrt{n}) + \log n$

14.2.1 Bước 1: Đổi biến

Đặt $n = 2^k$ (với $k = \log_2 n$), khi đó:

$$\sqrt{n} = 2^{k/2}$$

$$\log n = \log_2(2^k) = k$$

Đặt $S(k) = T(2^k) = T(n)$. Hệ thức trở thành:

$$S(k) = 2 \cdot S\left(\frac{k}{2}\right) + k$$

14.2.2 Bước 2: Áp dụng Master Theorem cho $S(k)$

Hệ thức:

$$S(k) = 2 \cdot S\left(\frac{k}{2}\right) + k$$

Ở đây:

- $a = 2, b = 2$
- $f(k) = k$
- $\log_b a = \log_2 2 = 1$
- $k^{\log_b a} = k^1 = k$

So sánh $f(k) = k$ với $k^{\log_b a} = k$:

$$f(k) = \Theta(k) = \Theta(k^{\log_b a})$$

Đây là **Trường hợp 2** của Master Theorem (với $k = 0$ trong $\log^k k$):

$$S(k) = \Theta(k^{\log_b a} \cdot \log^{k+1} k) = \Theta(k \cdot \log k)$$

14.2.3 Bước 3: Chuyển về biến n

Vì $k = \log_2 n$, ta có:

$$S(k) = \Theta(k \log k) = \Theta(\log n \cdot \log \log n)$$

Vậy:

$$\boxed{T(n) = \Theta(\log n \cdot \log \log n)}$$

14.3 Câu (b): $T(n) = T(\sqrt{n}) + 1$

14.3.1 Bước 1: Đổi biến

Đặt $n = 2^k$, $\sqrt{n} = 2^{k/2}$, $S(k) = T(2^k)$. Hệ thức trở thành:

$$S(k) = S\left(\frac{k}{2}\right) + 1$$

14.3.2 Bước 2: Giải hệ thức $S(k) = S(k/2) + 1$

Đây là dạng truy hồi chia đôi với hằng số. Giải bằng khai triển hoặc Master Theorem.

Cách 1: Khai triển lặp

$$\begin{aligned} S(k) &= S\left(\frac{k}{2}\right) + 1 \\ &= \left[S\left(\frac{k}{4}\right) + 1\right] + 1 = S\left(\frac{k}{4}\right) + 2 \\ &= S\left(\frac{k}{8}\right) + 3 \\ &= \dots \\ &= S\left(\frac{k}{2^i}\right) + i \end{aligned}$$

Dừng khi $\frac{k}{2^i} = 1 \Rightarrow 2^i = k \Rightarrow i = \log_2 k$.

Vậy:

$$S(k) = S(1) + \log_2 k = \Theta(\log k)$$

Cách 2: Master Theorem Với $S(k) = 1 \cdot S(k/2) + 1$:

- $a = 1, b = 2, f(k) = 1$

- $\log_b a = \log_2 1 = 0$
- $k^{\log_b a} = k^0 = 1$
- So sánh: $f(k) = \Theta(1) = \Theta(k^{\log_b a})$ Trường hợp 2 với $k = 0$
- $S(k) = \Theta(k^{\log_b a} \cdot \log^{k+1} k) = \Theta(\log k)$

14.3.3 Bước 3: Chuyển về biến n

Vì $k = \log_2 n$:

$$S(k) = \Theta(\log k) = \Theta(\log \log n)$$

Vậy:

$$T(n) = \Theta(\log \log n)$$

14.4 Liên hệ giữa câu (a) và câu (b)

Có thể giải câu (b) bằng cách nhận xét: Câu (b) là trường hợp đặc biệt của câu (a) khi thay $a = 1$ và $f(n) = 1$? Không hoàn toàn, nhưng có một cách khác:

Nếu đặt $U(n) = T(n) - T(\sqrt{n})$, không đơn giản. Thay vào đó, ta có thể thấy rằng câu (b) có dạng tương tự câu (a) với hệ số 1 thay vì 2 và số hạng +1 thay vì $\log n$.

Tuy nhiên, kết quả câu (b) $\Theta(\log \log n)$ nhỏ hơn nhiều so với câu (a) $\Theta(\log n \cdot \log \log n)$, điều này hợp lý vì câu (a) có thêm hệ số nhân 2 làm tăng độ phức tạp.

14.5 Bảng tổng kết

Câu	Hệ thức	Sau đổi biến	Kết quả
(a)	$T(n) = 2T(\sqrt{n}) + \log n$	$S(k) = 2S(k/2) + k$	$\Theta(\log n \cdot \log \log n)$
(b)	$T(n) = T(\sqrt{n}) + 1$	$S(k) = S(k/2) + 1$	$\Theta(\log \log n)$

14.6 Bài tập tương tự

Hãy giải các hệ thức sau:

1. $T(n) = 4T(\sqrt{n}) + n$
2. $T(n) = 3T(\sqrt{n}) + \log^2 n$
3. $T(n) = T(\sqrt{n}) + \log \log n$

Đáp án gợi ý:

1. Đặt $n = 2^k$, được $S(k) = 4S(k/2) + 2^k$. Áp dụng Master Theorem: $\log_2 4 = 2$, $f(k) = 2^k$ lớn hơn k^2 , thuộc TH3? Thực tế cần so sánh 2^k với k^2 : 2^k lớn hơn, nhưng cần kiểm tra điều kiện chính quy. Kết quả: $\Theta(n)$.
2. Tương tự câu (a): $S(k) = 3S(k/2) + k^2$ $S(k) = \Theta(k^{\log_2 3}) = \Theta(k^{1.585})$ $T(n) = \Theta((\log n)^{1.585})$.
3. $S(k) = S(k/2) + \log k$ $S(k) = \Theta(\log^2 k)$ $T(n) = \Theta((\log \log n)^2)$.

15 Bài tập 2

15.1 Bài toán 1: Tìm phần tử lớn nhất và nhỏ nhất trong mảng

Cho một mảng số nguyên, hãy tìm cả phần tử lớn nhất và nhỏ nhất bằng phương pháp chia để trị.

15.1.1 Thuật toán

```
0: function FINDMINMAX( $A, l, r$ )
0:   if  $l = r$  then
0:     return ( $A[l], A[l]$ )
0:   else if  $l + 1 = r$  then
0:     if  $A[l] < A[r]$  then
0:       return ( $A[l], A[r]$ )
0:     else
0:       return ( $A[r], A[l]$ )
0:     end if
0:   else
0:      $mid \leftarrow \lfloor (l + r) / 2 \rfloor$ 
0:     ( $min1, max1$ )  $\leftarrow$  FINDMINMAX( $A, l, mid$ )
0:     ( $min2, max2$ )  $\leftarrow$  FINDMINMAX( $A, mid + 1, r$ )
0:      $min\_all \leftarrow \min(min1, min2)$ 
0:      $max\_all \leftarrow \max(max1, max2)$ 
0:     return ( $min\_all, max\_all$ )
0:   end if
0: end function
```

15.1.2 Độ phức tạp

$$T(n) = 2T\left(\frac{n}{2}\right) + 2 = \Theta(n)$$

Số phép so sánh: $\frac{3n}{2} - 2$ (tốt hơn so với duyệt tuần tự $2n - 2$).

15.1.3 Mã Java (dàn ý nhanh)

```
public class MinMax {
    static class Pair { int min, max; }
    static Pair findMinMax(int[] arr, int l, int r) {
        if (l == r) return new Pair(arr[l], arr[l]);
        if (l + 1 == r) {
            if (arr[l] < arr[r]) return new Pair(arr[l], arr[r]);
            else return new Pair(arr[r], arr[l]);
        }
        int mid = (l + r) / 2;
        Pair left = findMinMax(arr, l, mid);
        Pair right = findMinMax(arr, mid + 1, r);
        return new Pair(Math.min(left.min, right.min),
                        Math.max(left.max, right.max));
    }
}
```

15.2 Bài toán 2: Nhân ma trận (Chia để trị chuẩn và Strassen)

Cho hai ma trận vuông A và B cấp n (với n là lũy thừa của 2), tính $C = A \times B$.

15.2.1 2.1 Chia để trị chuẩn

Phân hoạch mỗi ma trận thành 4 ma trận con kích thước $n/2$:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}, \quad C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

Tính:

$$C_{11} = A_{11}B_{11} + A_{12}B_{21}$$

$$C_{12} = A_{11}B_{12} + A_{12}B_{22}$$

$$C_{21} = A_{21}B_{11} + A_{22}B_{21}$$

$$C_{22} = A_{21}B_{12} + A_{22}B_{22}$$

15.2.2 Độ phức tạp

$$T(n) = 8T\left(\frac{n}{2}\right) + O(n^2) \Rightarrow T(n) = O(n^3)$$

15.2.3 2.2 Thuật toán Strassen

Strassen chỉ dùng 7 phép nhân thay vì 8:

Tính:

$$M_1 = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$M_2 = (A_{21} + A_{22})B_{11}$$

$$M_3 = A_{11}(B_{12} - B_{22})$$

$$M_4 = A_{22}(B_{21} - B_{11})$$

$$M_5 = (A_{11} + A_{12})B_{22}$$

$$M_6 = (A_{21} - A_{11})(B_{11} + B_{12})$$

$$M_7 = (A_{12} - A_{22})(B_{21} + B_{22})$$

Sau đó:

$$C_{11} = M_1 + M_4 - M_5 + M_7$$

$$C_{12} = M_3 + M_5$$

$$C_{21} = M_2 + M_4$$

$$C_{22} = M_1 - M_2 + M_3 + M_6$$

15.2.4 Độ phức tạp của Strassen

$$T(n) = 7T\left(\frac{n}{2}\right) + O(n^2) \Rightarrow T(n) = O(n^{\log_2 7}) \approx O(n^{2.81})$$

15.3 Bài toán 3: Sắp xếp nhanh (QuickSort)

Sắp xếp một mảng theo thứ tự tăng dần hoặc giảm dần bằng QuickSort.

15.3.1 Thuật toán (tăng dần)

```
0: function QUICKSORT(A, low, high)
0:   if low < high then
0:     pivot_index ← PARTITION(A, low, high)
0:     QUICKSORT(A, low, pivot_index - 1)
0:     QUICKSORT(A, pivot_index + 1, high)
0:   end if
0: end function=0
```

15.3.2 Phân hoạch (Lomuto)

```
0: function PARTITION(A, low, high)
0:   pivot ← A[high]
0:   i ← low - 1
0:   for j = low to high - 1 do
0:     if A[j] ≤ pivot then
0:       i ← i + 1
0:       swap A[i] and A[j]
0:     end if
0:   end for
0:   swap A[i + 1] and A[high]
0:   return i + 1
0: end function=0
```

15.3.3 Độ phức tạp

- Tốt nhất / Trung bình: $O(n \log n)$
- Tệ nhất: $O(n^2)$ (khi mảng đã được sắp xếp)

15.3.4 Đối với thứ tự giảm dần

Thay đổi điều kiện thành $A[j] \geq pivot$.

15.4 Bài toán 4: Sắp xếp trộn (MergeSort)

Sắp xếp một mảng theo thứ tự tăng dần hoặc giảm dần bằng MergeSort.

15.4.1 Thuật toán (tăng dần)

```
0: function MERGESORT( $A, l, r$ )
0:   if  $l < r$  then
0:      $mid \leftarrow \lfloor (l + r) / 2 \rfloor$ 
0:     MERGESORT( $A, l, mid$ )
0:     MERGESORT( $A, mid + 1, r$ )
0:     MERGE( $A, l, mid, r$ )
0:   end if
0: end function=0
```

15.4.2 Hàm trộn (Merge)

```
0: function MERGE( $A, l, mid, r$ )
0:    $n1 \leftarrow mid - l + 1$ 
0:    $n2 \leftarrow r - mid$ 
0:   tạo mảng  $L[0..n1 - 1], R[0..n2 - 1]$ 
0:   for  $i = 0$  to  $n1 - 1$  do
0:      $L[i] \leftarrow A[l + i]$ 
0:   end for
0:   for  $j = 0$  to  $n2 - 1$  do
0:      $R[j] \leftarrow A[mid + 1 + j]$ 
0:   end for
0:    $i \leftarrow 0, j \leftarrow 0, k \leftarrow l$ 
0:   while  $i < n1$  and  $j < n2$  do
0:     if  $L[i] \leq R[j]$  then
0:        $A[k] \leftarrow L[i]; i \leftarrow i + 1$ 
0:     else
0:        $A[k] \leftarrow R[j]; j \leftarrow j + 1$ 
0:     end if
0:      $k \leftarrow k + 1$ 
0:   end while
0:   sao chép phần còn lại của  $L$  hoặc  $R$ 
0: end function=0
```

15.4.3 Độ phức tạp

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n) \Rightarrow T(n) = O(n \log n)$$

Luôn là $O(n \log n)$ bất kể thứ tự đầu vào.

15.4.4 Đối với thứ tự giảm dần

Thay đổi điều kiện thành $L[i] \geq R[j]$.

15.5 Bài toán 5: Kiểm tra phần tử xuất hiện đa số trong mảng đã sắp xếp

Cho một mảng đã được sắp xếp, hãy tìm xem có tồn tại phần tử nào xuất hiện nhiều hơn $n/2$ lần hay không.

15.5.1 Nhận xét

Trong mảng đã sắp xếp, nếu phần tử đa số tồn tại thì nó phải là phần tử ở giữa (tại chỉ số $n/2$).

15.5.2 Thuật toán

```
0: function HASMAJORITY( $A, n$ )
0:    $candidate \leftarrow A[n/2]$ 
0:    $count \leftarrow 0$ 
0:   for  $i = 0$  to  $n - 1$  do
0:     if  $A[i] = candidate$  then
0:        $count \leftarrow count + 1$ 
0:     end if
0:   end for
0:   if  $count > n/2$  then
0:     return true
0:   else
0:     return false
0:   end if
0: end function=0
```

15.5.3 Cách tiếp cận chia để trị (cho mảng chưa sắp xếp)

```
0: function FINDMAJORITY( $A, l, r$ )
0:   if  $l = r$  then
0:     return  $A[l]$ 
0:   end if
0:    $mid \leftarrow \lfloor (l + r)/2 \rfloor$ 
0:    $left \leftarrow$  FINDMAJORITY( $A, l, mid$ )
0:    $right \leftarrow$  FINDMAJORITY( $A, mid + 1, r$ )
0:   if  $left = right$  then
0:     return  $left$ 
0:   end if
0:    $left\_count \leftarrow$  COUNT( $A, l, r, left$ )
0:    $right\_count \leftarrow$  COUNT( $A, l, r, right$ )
0:   if  $left\_count > (r - l + 1)/2$  then
0:     return  $left$ 
0:   else if  $right\_count > (r - l + 1)/2$  then
0:     return  $right$ 
0:   else
0:     return null
0:   end if
0: end function=0
```

15.5.4 Độ phức tạp

- Mảng đã sắp xếp: $O(n)$
- Chia để trị cho mảng chưa sắp xếp: $O(n \log n)$

15.6 Bài toán 6: Trung vị của hai mảng đã sắp xếp có kích thước khác nhau

Cho hai mảng đã sắp xếp A (kích thước m) và B (kích thước n), tìm trung vị của mảng hợp đã được sắp xếp mà không cần trộn toàn bộ chúng.

15.6.1 Định nghĩa

Nếu tổng độ dài $m + n$ lẻ, trung vị là phần tử ở giữa. Nếu chẵn, trung vị là trung bình của hai phần tử giữa.

15.6.2 Thuật toán chia để trị

```
0: function FINDMEDIAN( $A, B$ )
0:   if  $|A| > |B|$  then
0:     swap  $A$  and  $B$ 
0:   end if
0:    $x \leftarrow |A|, y \leftarrow |B|$ 
0:    $low \leftarrow 0, high \leftarrow x$ 
0:   while  $low \leq high$  do
0:      $partitionX \leftarrow (low + high)/2$ 
0:      $partitionY \leftarrow (x + y + 1)/2 - partitionX$ 
0:      $maxLeftX \leftarrow (partitionX = 0)? -\infty : A[partitionX - 1]$ 
0:      $minRightX \leftarrow (partitionX = x)? \infty : A[partitionX]$ 
0:      $maxLeftY \leftarrow (partitionY = 0)? -\infty : B[partitionY - 1]$ 
0:      $minRightY \leftarrow (partitionY = y)? \infty : B[partitionY]$ 
0:     if  $maxLeftX \leq minRightY$  and  $maxLeftY \leq minRightX$  then
0:       if  $(x + y)$  lẻ then
0:         return  $\max(maxLeftX, maxLeftY)$ 
0:       else
0:         return  $\frac{\max(maxLeftX, maxLeftY) + \min(minRightX, minRightY)}{2}$ 
0:       end if
0:     else if  $maxLeftX > minRightY$  then
0:        $high \leftarrow partitionX - 1$ 
0:     else
0:        $low \leftarrow partitionX + 1$ 
0:     end if
0:   end while
0: end function
```

15.6.3 Độ phức tạp

$$O(\log(\min(m, n)))$$

15.6.4 Ví dụ

$A = [1, 3, 8], B = [7, 9, 10, 12]$. Mảng hợp = $[1, 3, 7, 8, 9, 10, 12]$, trung vị = 8.

15.7 Bảng tổng kết

Bài toán	Độ phức tạp thời gian
1. Min & Max	$O(n)$
2. Nhân ma trận chuẩn	$O(n^3)$
2. Strassen	$O(n^{2.81})$
3. QuickSort	Trung bình $O(n \log n)$, tệ $O(n^2)$
4. MergeSort	$O(n \log n)$
5. Phần tử đa số (đã sắp xếp)	$O(n)$
5. Phần tử đa số (chia để trị)	$O(n \log n)$
6. Trung vị hai mảng đã sắp xếp	$O(\log(\min(m, n)))$

16 Bài tập 3

16.1 Bài toán 1: Cặp điểm gần nhất (phiên bản 1D)

16.1.1 1a. Phát biểu bài toán

Cho một tập gồm n số thực $x_1, x_2, \dots, x_n$, tìm hai số có hiệu nhỏ nhất.

16.1.2 1b. Thiết kế thuật toán (Chia để trị)

Ý tưởng:

1. Sắp xếp mảng (nếu chưa được sắp xếp) – $O(n \log n)$.
2. Chia mảng đã sắp xếp thành hai nửa.
3. Đệ quy tìm cặp gần nhất trong nửa trái và nửa phải.
4. Cặp gần nhất có thể nằm giữa hai nửa: kiểm tra cặp $(mid, mid + 1)$.

Thuật toán:

```
0: function CLOSESTPAIR1D( $A, l, r$ )
0:   if  $l \geq r$  then
0:     return  $\infty$ 
0:   else if  $l + 1 = r$  then
0:     return  $A[r] - A[l]$ 
0:   else
0:      $mid \leftarrow \lfloor (l + r) / 2 \rfloor$ 
0:      $d\_left \leftarrow \text{CLOSESTPAIR1D}(A, l, mid)$ 
0:      $d\_right \leftarrow \text{CLOSESTPAIR1D}(A, mid + 1, r)$ 
0:      $d \leftarrow \min(d\_left, d\_right)$ 
0:      $d\_cross \leftarrow A[mid + 1] - A[mid]$  {bằng qua giữa}
0:     return  $\min(d, d\_cross)$ 
0:   end if
0: end function=0
```

16.1.3 1c. Phân tích độ phức tạp

Không có sắp xếp: đệ quy trên mảng chưa sắp xếp không hiệu quả. Với sắp xếp trước:

$$T_{\text{sort}}(n) = O(n \log n)$$

Sau đó đệ quy:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(1) \Rightarrow T(n) = O(n)$$

Tổng cộng: $O(n \log n) + O(n) = O(n \log n)$.

16.1.4 1d. Đây có phải thuật toán tốt không?

Trả lời: Không, nó không tốt cho bài toán này.

Lý do:

- Bài toán cặp điểm gần nhất 1D có thể giải bằng cách sắp xếp rồi duyệt tuyến tính: $O(n \log n)$.
- Thậm chí tốt hơn: sau khi sắp xếp, chỉ cần duyệt một lần: tổng $O(n \log n)$.
- Chia để trị thêm độ phức tạp không cần thiết.
- Một thuật toán đơn giản: sắp xếp + duyệt tuyến tính vừa đơn giản vừa hiệu quả.

Thuật toán tốt hơn:

```

1. Sắp xếp mảng:  $O(n \log n)$ 
2. min_dist = vô cùng
3. for i = 0 to n-2:
    min_dist = min(min_dist, A[i+1] - A[i])
4. return min_dist

```

Vậy đối với 1D, chia để trị là không cần thiết.

16.2 Bài toán 2: Bài toán đồng xu giả

16.2.1 2a. Phát biểu bài toán

Có n đồng xu, tất cả đều giống nhau về hình thức. Chính xác một đồng là giả (fake) và nhẹ hơn đồng thật. Bạn có một cân đĩa (không phải cân bàn). Hãy tìm đồng xu giả với số lần cân ít nhất.

16.2.2 2b. Thiết kế thuật toán (Giảm để trị)

Đây là bài toán **giảm theo hằng số nhân** (tìm kiếm tam phân).

Ý tưởng:

- Chia các đồng xu thành 3 nhóm càng đều càng tốt.
- Cân hai nhóm với nhau.
- Nếu cân bằng, đồng giả nằm trong nhóm thứ ba.
- Nếu không cân bằng, bên nhẹ hơn chứa đồng giả.
- Lặp lại trên nhóm nhỏ hơn.

Thuật toán:

```

0: function FINDFAKECOIN(coins[], l, r)
0:    $n \leftarrow r - l + 1$ 
0:   if  $n = 1$  then
0:     return l
0:   else if  $n = 2$  then
0:     Cân coins[l] vs coins[l + 1]
0:     return chỉ số của đồng nhẹ hơn
0:   else
0:      $groupSize \leftarrow \lfloor n/3 \rfloor$ 
0:      $g1 \leftarrow [l, l + groupSize - 1]$ 
0:      $g2 \leftarrow [l + groupSize, l + 2 * groupSize - 1]$ 
0:      $g3 \leftarrow [l + 2 * groupSize, r]$ 
0:     Cân g1 vs g2
0:     if g1 và g2 cân bằng then
0:       return FINDFAKECOIN(coins, start(g3), end(g3))
0:     else if g1 nhẹ hơn then
0:       return FINDFAKECOIN(coins, start(g1), end(g1))
0:     else
0:       return FINDFAKECOIN(coins, start(g2), end(g2))
0:     end if
0:   end if
0: end function=0

```

16.2.3 2c. Phân tích độ phức tạp

Mỗi bước, kích thước bài toán giảm xuống nhiều nhất $\lceil n/3 \rceil$:

$$T(n) = T\left(\frac{n}{3}\right) + O(1)$$

Giải:

$$T(n) = O(\log_3 n)$$

Số lần cân: $\lceil \log_3 n \rceil$.

16.2.4 2d. So sánh với cách tiếp cận nhị phân

Nếu chia thành 2 nhóm (giảm một nửa):

$$T(n) = T\left(\frac{n}{2}\right) + O(1) = O(\log_2 n)$$

Tam phân tốt hơn: $\log_3 n < \log_2 n$.

16.2.5 Ví dụ

Với $n = 27$ đồng xu:

- Phương pháp nhị phân: $\lceil \log_2 27 \rceil = 5$ lần cân.
- Phương pháp tam phân: $\lceil \log_3 27 \rceil = 3$ lần cân.

16.2.6 2e. Cài đặt minh họa (Python)

```
def find_fake_coin(coins, left, right):
    n = right - left + 1
    if n == 1:
        return left
    if n == 2:
        return left if coins[left] < coins[right] else right

    group_size = n // 3
    # cân nhóm1 vs nhóm2
    sum1 = sum(coins[left:left+group_size])
    sum2 = sum(coins[left+group_size:left+2*group_size])

    if sum1 == sum2:
        return find_fake_coin(coins, left+2*group_size, right)
    elif sum1 < sum2:
        return find_fake_coin(coins, left, left+group_size-1)
    else:
        return find_fake_coin(coins, left+group_size, left+2*group_size-1)
```

16.3 Bảng tổng kết

Bài toán	Mô hình	Hệ thức	Độ phức tạp
Cặp điểm gần nhất 1D (D&C)	Chia để trị	$T(n) = 2T(n/2) + O(1)$	$O(n \log n)$
Cặp điểm gần nhất 1D (tốt hơn)	Sắp xếp + duyệt	—	$O(n \log n)$
Đồng xu giả (tam phân)	Giảm theo hệ số	$T(n) = T(n/3) + O(1)$	$O(\log_3 n)$
Đồng xu giả (nhị phân)	Giảm theo hệ số	$T(n) = T(n/2) + O(1)$	$O(\log_2 n)$

16.4 Kết luận

- Đối với cặp điểm gần nhất 1D, chia để trị hoạt động nhưng không cần thiết; sắp xếp + duyệt tuyến tính đơn giản hơn và hiệu quả tương đương.
- Đối với bài toán đồng xu giả, chiến lược giảm theo hệ số tam phân là tối ưu, đạt $\lceil \log_3 n \rceil$ lần cân.
- Cả hai bài toán minh họa cách lựa chọn mô hình (chia để trị hay giảm để trị) ảnh hưởng đến thiết kế và hiệu quả của thuật toán.

17 Bài tập 4

17.1 Phát biểu bài toán

Cho một mảng $P[0..n-1]$ biểu diễn giá cổ phiếu trên n ngày liên tiếp, trong đó:

- $P[i]$ là giá vào ngày i .
- Các giá không hoàn toàn giống nhau.
- Bạn có thể mua nhiều nhất một lần và bán nhiều nhất một lần.
- Bạn phải mua trước khi bán.

Tìm các chỉ số i (ngày mua) và j (ngày bán) với $i < j$ để tối đa hóa:

$$\text{Lợi nhuận} = P[j] - P[i]$$

17.2 Cách tiếp cận chia để trị

17.2.1 Ý tưởng

Lợi nhuận lớn nhất có thể xảy ra theo một trong ba cách:

1. Mua và bán hoàn toàn trong **nửa trái** của mảng.
2. Mua và bán hoàn toàn trong **nửa phải** của mảng.
3. Mua ở **nửa trái** và bán ở **nửa phải**.

17.2.2 Thuật toán

```

0: function MAXPROFITDAC( $P, l, r$ )
0:   if  $l = r$  then
0:     return  $(0, l, l)$  {không thể có lợi nhuận}
0:   end if
0:    $mid \leftarrow \lfloor (l + r) / 2 \rfloor$ 
0:    $(profit\_left, buy\_left, sell\_left) \leftarrow \text{MAXPROFITDAC}(P, l, mid)$ 
0:    $(profit\_right, buy\_right, sell\_right) \leftarrow \text{MAXPROFITDAC}(P, mid + 1, r)$ 
0:    $(profit\_cross, buy\_cross, sell\_cross) \leftarrow \text{MAXCROSSINGPROFIT}(P, l, mid, r)$ 
0:   return bộ có lợi nhuận lớn nhất trong ba bộ
0: end function

```

17.2.3 Lợi nhuận băng qua

Để tìm lợi nhuận lớn nhất khi mua ở nửa trái và bán ở nửa phải:

- Tìm giá nhỏ nhất trong nửa trái (ngày tốt nhất để mua).
- Tìm giá lớn nhất trong nửa phải (ngày tốt nhất để bán).

```

0: function MAXCROSSINGPROFIT( $P, l, mid, r$ )
0:    $min\_price \leftarrow P[mid]$ 
0:    $buy\_index \leftarrow mid$ 
0:   for  $i = mid$  downto  $l$  do
0:     if  $P[i] < min\_price$  then
0:        $min\_price \leftarrow P[i]$ 
0:        $buy\_index \leftarrow i$ 
0:     end if
0:   end for
0:    $max\_price \leftarrow P[mid + 1]$ 
0:    $sell\_index \leftarrow mid + 1$ 
0:   for  $j = mid + 1$  to  $r$  do
0:     if  $P[j] > max\_price$  then
0:        $max\_price \leftarrow P[j]$ 
0:        $sell\_index \leftarrow j$ 
0:     end if
0:   end for
0:   return  $(max\_price - min\_price, buy\_index, sell\_index)$ 
0: end function

```


17.3 Phân tích độ phức tạp

Hệ thức:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

Với $O(n)$ là chi phí tính lợi nhuận bằng qua (duyệt nửa trái và nửa phải).

Theo Định lý Thợ (Trường hợp 2):

$$T(n) = \Theta(n \log n)$$

17.4 So sánh với các cách tiếp cận khác

17.4.1 Vét cạn

Thử tất cả các cặp (i, j) :

$$T(n) = \binom{n}{2} = O(n^2)$$

17.4.2 Thuật toán Kadane (Quy hoạch động - Tối ưu)

Ý tưởng: giữ giá nhỏ nhất cho đến hiện tại:

```
min_price = vô cùng
max_profit = 0
for each price in prices:
    min_price = min(min_price, price)
    max_profit = max(max_profit, price - min_price)
```

Độ phức tạp: $O(n)$ thời gian, $O(1)$ không gian.

17.4.3 Bảng so sánh

Phương pháp	Thời gian	Không gian
Vét cạn	$O(n^2)$	$O(1)$
Chia để trị	$O(n \log n)$	$O(\log n)$ (ngăn xếp đệ quy)
Kadane	$O(n)$	$O(1)$

17.5 Chia để trị có phải là lựa chọn tốt không?

- **Có cho việc học:** Nó minh họa rõ ràng mô hình chia để trị.
- **Không cho thực hành:** Thuật toán Kadane đơn giản hơn và nhanh hơn ($O(n)$ so với $O(n \log n)$).
- Bước tính lợi nhuận bằng qua thực chất là duyệt, khá đơn giản.
- Tuy nhiên, D&C vẫn đủ hiệu quả cho n lớn (ví dụ $n = 10^6$ vẫn ổn).

17.6 Cài đặt đầy đủ (Python)

```
def max_profit_dac(prices, left, right):
    if left == right:
        return (0, left, left)

    mid = (left + right) // 2

    # Nửa trái
    profit_left, buy_left, sell_left = max_profit_dac(prices, left, mid)

    # Nửa phải
    profit_right, buy_right, sell_right = max_profit_dac(prices, mid + 1, right)

    # Băng qua
```

```

# Tìm min trong nửa trái
min_price = prices[mid]
buy_cross = mid
for i in range(mid, left - 1, -1):
    if prices[i] < min_price:
        min_price = prices[i]
        buy_cross = i

# Tìm max trong nửa phải
max_price = prices[mid + 1]
sell_cross = mid + 1
for j in range(mid + 1, right + 1):
    if prices[j] > max_price:
        max_price = prices[j]
        sell_cross = j

profit_cross = max_price - min_price

# Trả về tốt nhất
if profit_left >= profit_right and profit_left >= profit_cross:
    return (profit_left, buy_left, sell_left)
elif profit_right >= profit_left and profit_right >= profit_cross:
    return (profit_right, buy_right, sell_right)
else:
    return (profit_cross, buy_cross, sell_cross)

# Phiên bản Kadane tối ưu
def max_profit_kadane(prices):
    if not prices:
        return (0, -1, -1)
    min_price = prices[0]
    min_index = 0
    max_profit = 0
    buy_day = 0
    sell_day = 0

    for i in range(1, len(prices)):
        if prices[i] < min_price:
            min_price = prices[i]
            min_index = i
        profit = prices[i] - min_price
        if profit > max_profit:
            max_profit = profit
            buy_day = min_index
            sell_day = i
    return (max_profit, buy_day, sell_day)

# Ví dụ
prices = [100, 180, 260, 310, 40, 535, 695]
profit, buy, sell = max_profit_kadane(prices)
print(f"Mua ngày {buy}, bán ngày {sell}, lợi nhuận = {profit}")

```

17.7 Ví dụ minh họa

Cho $P = [100, 180, 260, 310, 40, 535, 695]$.

- Vết cặn: kiểm tra tất cả các cặp $\rightarrow$ lớn nhất là $695 - 40 = 655$ (mua ngày 4, bán ngày 6).
- Chia để trị:

- Nửa trái [100, 180, 260, 310]: lợi nhuận lớn nhất = 210 (mua 100, bán 310).
- Nửa phải [40, 535, 695]: lợi nhuận lớn nhất = 655 (mua 40, bán 695).
- Băng qua: min trái = 40, max phải = 695 $\rightarrow$ lợi nhuận = 655.
- Tốt nhất = 655.

- Kadane: min_price theo dõi 100 $\rightarrow$ 40, max_profit trở thành 655.

17.8 Kết luận

- Lời giải chia để trị chạy trong thời gian $O(n \log n)$.
- Nó đúng đắn và minh họa mô hình ba trường hợp (trái, phải, băng qua).
- Tuy nhiên, lời giải tối ưu là thuật toán Kadane với $O(n)$.
- Bài toán này là ví dụ kinh điển cho thấy cách tiếp cận đơn giản hơn (quy hoạch động) vượt trội so với chia để trị.

18 Bài tập 5

18.1 Phát biểu bài toán

Chúng ta có một tòa nhà n tầng (đánh số từ 1 đến n từ dưới lên). Có hai chiếc laptop giống hệt nhau.

Mục tiêu: Tìm tầng cao nhất mà từ đó thả laptop xuống **không bị vỡ**, gọi là "ngưỡng bền" (ceiling).

Tính chất: - Nếu laptop vỡ khi thả từ tầng f , nó sẽ vỡ khi thả từ mọi tầng $\geq f$. - Nếu laptop không vỡ khi thả từ tầng f , nó sẽ không vỡ khi thả từ mọi tầng $\leq f$. - Một laptop bị vỡ thì không thể dùng lại.

Yêu cầu: Tìm thuật toán **tối ưu số lần thử** trong trường hợp xấu nhất (worst-case), với 2 laptop.

18.2 Ý tưởng chính

Với 1 laptop: chỉ có cách thử từng tầng từ dưới lên $\rightarrow n$ lần thử trong worst case.

Với 2 laptop: ta có thể "hy sinh" laptop thứ nhất để thu hẹp khoảng tìm kiếm, sau đó dùng laptop thứ hai để thử chi tiết.

Chiến lược tối ưu: **thử với bước nhảy giảm dần** sao cho tổng số lần thử trong mọi trường hợp là như nhau (hoặc xấp xỉ).

18.3 Phân tích toán học

Giả sử ta thực hiện lần thử đầu tiên ở tầng x . Có hai trường hợp:

1. **Laptop vỡ:** Mất laptop thứ nhất. Phải dùng laptop thứ hai thử từ tầng 1 đến $x - 1$ (từng tầng một) $\rightarrow$ tối đa $x - 1$ lần thử nữa. Tổng số lần thử trong trường hợp này: $1 + (x - 1) = x$.
2. **Laptop không vỡ:** Vẫn còn cả hai laptop. Vấn đề trở thành tìm ngưỡng trong khoảng từ $x + 1$ đến n với 2 laptop. Nếu ta chọn bước nhảy không đổi, số lần thử sẽ khác nhau giữa các nhánh.

Để tối ưu hóa trường hợp xấu nhất, ta muốn **số lần thử trong mọi nhánh là như nhau** (hoặc xấp xỉ).

18.4 Xây dựng bước nhảy giảm dần

Gọi $f(n)$ là số lần thử tối thiểu trong worst case với n tầng và 2 laptop.

Giả sử lần thử đầu tiên ở tầng k . Khi đó: - Nếu vỡ: cần $k - 1$ lần thử tiếp (vì laptop 2 thử từng tầng 1..k-1) $\rightarrow$ tổng k lần. - Nếu không vỡ: còn lại $n - k$ tầng với 2 laptop $\rightarrow$ cần $f(n - k)$ lần nữa.

Vậy:

$$f(n) = \min_{1 \leq k \leq n} \max(k, 1 + f(n - k))$$

Đây là công thức quy hoạch động, nhưng ta có thể tìm dạng đóng.

18.5 Chiến lược bước nhảy giảm dần

Ta chọn các tầng thử: $t_1, t_2, t_3, \dots$ sao cho: - Lần 1: thử tầng d - Lần 2: thử tầng $d + (d - 1)$ - Lần 3: thử tầng $d + (d - 1) + (d - 2) - \dots$ Sao cho tổng bước nhảy phủ hết n tầng.

Tức là:

$$d + (d - 1) + (d - 2) + \dots + 1 \geq n$$

$$\frac{d(d+1)}{2} \geq n$$

Khi đó, số lần thử tối đa trong worst case là d .

18.6 Thuật toán

```
0: function FINDCEILING( $n$ )
0:   Tìm số nguyên  $d$  nhỏ nhất sao cho  $\frac{d(d+1)}{2} \geq n$ 
0:    $step \leftarrow d$ 
0:    $current \leftarrow 0$ 
0:   {Dùng laptop 1 để nhảy bước giảm dần}
0:   while  $step \geq 1$  do
0:      $current \leftarrow current + step$ 
0:     if  $current > n$  then
0:        $current \leftarrow n$ 
0:     end if
0:     Thả laptop 1 từ tầng  $current$ 
0:     if laptop 1 vỡ then
0:       {Dùng laptop 2 thử từng tầng từ  $current - step + 1$  đến  $current - 1$ }
0:       for  $floor = current - step + 1$  to  $current - 1$  do
0:         Thả laptop 2 từ tầng  $floor$ 
0:         if laptop 2 vỡ then
0:           return  $floor - 1$  {ngưỡng bền là tầng liền dưới}
0:         end if
0:       end for
0:       return  $current - 1$  {không vỡ ở tầng  $current - 1$ }
0:     end if
0:      $step \leftarrow step - 1$ 
0:   end while
0:   return  $n$  {laptop không vỡ ở tầng cao nhất}
0: end function
```

18.7 Ví dụ minh họa

18.7.1 Ví dụ 1: $n = 100$ tầng

Cần tìm d nhỏ nhất sao cho:

$$\frac{d(d+1)}{2} \geq 100$$

Thử: - $d = 13$: $13 \cdot 14/2 = 91 < 100$ - $d = 14$: $14 \cdot 15/2 = 105 \geq 100$

Vậy $d = 14$.

Các tầng thử với laptop 1: 14, 27, 39, 50, 60, 69, 77, 84, 90, 95, 99, 100 (hoặc đến khi vỡ).

Worst case: laptop 1 vỡ ở lần cuối $\rightarrow$ mất 14 lần thử.

18.7.2 Ví dụ 2: $n = 36$ tầng

$$d = 8 \quad (8 \cdot 9/2 = 36)$$

Các tầng thử: 8, 15, 21, 26, 30, 33, 35, 36.

Worst case: 8 lần thử.

18.8 Phân tích độ phức tạp

Số lần thử tối đa:

$$f(n) = \left\lceil \frac{-1 + \sqrt{1 + 8n}}{2} \right\rceil$$

Do đó:

$$f(n) = \Theta(\sqrt{n})$$

So sánh: - 1 laptop: $O(n)$ - 2 laptop (thuật toán tối ưu): $O(\sqrt{n})$

18.9 Cài đặt minh họa (Python)

```
import math

def find_ceiling_two_laptops(n):
    # Tìm d nhỏ nhất sao cho d(d+1)/2 >= n
    d = math.ceil((math.sqrt(1 + 8*n) - 1) / 2)

    step = d
    current = 0
    laptop1_broken = False

    while step >= 1 and not laptop1_broken:
        current += step
        if current > n:
            current = n

        print(f"Thả laptop 1 ở tầng {current}")

        # Giả lập: giả sử ngưỡng bền là threshold (ví dụ 45)
        # Ở đây ta chỉ mô phỏng thuật toán
        threshold = 45 # giá trị ví dụ

        if current >= threshold: # laptop vỡ
            laptop1_broken = True
            print(f" Laptop 1 vỡ ở tầng {current}")
            # Dùng laptop 2 thử từ current - step + 1 đến current - 1
            for floor in range(current - step + 1, current):
                print(f" Thả laptop 2 ở tầng {floor}")
                if floor >= threshold:
                    print(f" Laptop 2 vỡ ở tầng {floor}")
                    return floor - 1
            return current - 1
        else:
            print(f" Laptop 1 không vỡ ở tầng {current}")

        step -= 1

    return n

# Chạy thử
n = 100
result = find_ceiling_two_laptops(n)
print(f"Ngưỡng bền là tầng: {result}")
```

18.10 Kết luận

- Với 2 laptop, ta có thể tìm ngưỡng bền trong $O(\sqrt{n})$ lần thử.
- Chiến lược tối ưu là **thử với bước nhảy giảm dần**: $d, d-1, d-2, \dots, 1$ sao cho tổng $\geq n$.

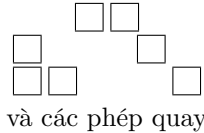
- Thuật toán này đảm bảo số lần thử trong worst case là nhỏ nhất có thể.
- Đây là một ứng dụng kinh điển của **chiến lược giảm để trị (decrease and conquer)**.

19 Bài tập 6

19.1 Phát biểu bài toán

Cho một sàn hình vuông kích thước $2^n \times 2^n$ ô (với $n \geq 1$). Một ô được chọn làm **ô thoát nước (drainage)** – ô này sẽ không được lát gạch.

Có sẵn các viên gạch hình chữ L (tromino) gồm 3 ô, có dạng:



Yêu cầu: Tìm cách lát toàn bộ sàn (trừ ô thoát nước) bằng các viên gạch hình chữ L, không chồng lấn, không để trống ô nào khác.

19.2 Ý tưởng chia để trị

Đây là bài toán kinh điển áp dụng **chia để trị (divide and conquer)**.

19.2.1 Tư tưởng chính

1. Chia sàn $2^n \times 2^n$ thành 4 sàn con, mỗi sàn có kích thước $2^{n-1} \times 2^{n-1}$.
2. Ô thoát nước nằm ở một trong 4 góc phần tư.
3. Đặt **một viên gạch hình chữ L** ở chính giữa sàn lớn, phủ lên 3 ô trung tâm của 3 góc phần tư **không chứa ô thoát nước**.
4. Sau khi đặt viên gạch trung tâm đó, mỗi góc phần tư đều có **đúng một ô đã được lát sẵn** (hoặc là ô thoát nước, hoặc là ô thuộc viên gạch trung tâm).
5. Đệ quy giải bài toán lát cho từng góc phần tư với ô đặc biệt riêng của nó.

19.2.2 Minh họa

Với $n = 2$ (sàn 4×4), ô thoát nước ở góc trên bên trái:

```
X . . .
. . . .
. . . .
. . . .
```

Đặt viên gạch trung tâm che 3 ô ở giữa của 3 góc còn lại:

```
X . . .
. T T .
. T . .
. . . .
```

Sau đó mỗi góc phần tư có 1 ô đã được lát (X hoặc T), tiếp tục lát đệ quy.

19.3 Thuật toán

```

0: function TILE(board, n, row, col, missing_row, missing_col) {n là lũy thừa của 2, kích thước sàn
    hiện tại là  $n \times n$ } {(row, col) là góc trên bên trái của sàn hiện tại} {(missing_row, missing_col) là
    ô trống (thoát nước) trong sàn này}
0:   if n = 1 then
0:     return
0:   end if
0:   half  $\leftarrow n/2$ 
0:   Xác định góc phần tư chứa ô trống:
0:   center_row  $\leftarrow row + half - 1$ 
0:   center_col  $\leftarrow col + half - 1$ 
0:   {Đặt viên gạch chữ L ở trung tâm, che 3 ô của 3 góc không chứa ô trống}
0:   if ô trống ở góc trên bên trái then
0:     Đặt gạch tại (center_row+1, center_col), (center_row, center_col+1), (center_row+1, center_col+
1)
0:     TILE(board, half, row, col, missing_row, missing_col)
0:     TILE(board, half, row, col + half, row + half - 1, col + half)
0:     TILE(board, half, row + half, col, row + half, col + half - 1)
0:     TILE(board, half, row + half, col + half, row + half, col + half)
0:   else if ô trống ở góc trên bên phải then
0:     Đặt gạch tại (center_row, center_col), (center_row+1, center_col), (center_row+1, center_col+
1)
0:     TILE(board, half, row, col, row + half - 1, col + half - 1)
0:     TILE(board, half, row, col + half, missing_row, missing_col)
0:     TILE(board, half, row + half, col, row + half, col + half - 1)
0:     TILE(board, half, row + half, col + half, row + half, col + half)
0:   else if ô trống ở góc dưới bên trái then
0:     Đặt gạch tại (center_row, center_col), (center_row, center_col+1), (center_row+1, center_col+
1)
0:     TILE(board, half, row, col, row + half - 1, col + half - 1)
0:     TILE(board, half, row, col + half, row + half - 1, col + half)
0:     TILE(board, half, row + half, col, missing_row, missing_col)
0:     TILE(board, half, row + half, col + half, row + half, col + half)
0:   else{ô trống ở góc dưới bên phải}
0:     Đặt gạch tại (center_row, center_col), (center_row, center_col+1), (center_row+1, center_col)
0:     TILE(board, half, row, col, row + half - 1, col + half - 1)
0:     TILE(board, half, row, col + half, row + half - 1, col + half)
0:     TILE(board, half, row + half, col, row + half, col + half - 1)
0:     TILE(board, half, row + half, col + half, missing_row, missing_col)
0:   end if
0: end function=0

```

19.4 Phân tích độ phức tạp

Gọi $T(n)$ là số viên gạch cần dùng (hoặc số thao tác) với sàn kích thước $2^n \times 2^n$.

Mỗi bước chia làm 4 bài toán con kích thước $2^{n-1} \times 2^{n-1}$, và đặt 1 viên gạch trung tâm.

$$T(n) = 4 \cdot T(n-1) + 1$$

Với $T(1) = 1$ (sàn 2×2 thiếu 1 ô $\rightarrow$ cần 1 viên gạch).

Giải:

$$T(n) = 4^{n-1} \cdot T(1) + (4^{n-2} + 4^{n-3} + \dots + 1)$$

$$T(n) = 4^{n-1} + \frac{4^{n-1} - 1}{3} = \frac{4^n - 1}{3}$$

Số ô trên sàn: 4^n . Bỏ 1 ô trống, còn $4^n - 1$ ô. Mỗi viên gạch che 3 ô, nên số gạch là:

$$\frac{4^n - 1}{3}$$

Đúng với kết quả trên.

Vậy thuật toán chạy trong thời gian $O(4^n)$ – tỷ lệ thuận với số ô, là tối ưu vì phải đặt từng viên gạch.

19.5 Ví dụ minh họa

$n = 1$: sàn 2×2

X . hoặc . X hoặc . . hoặc . .
. . . . X . . X

Chỉ cần 1 viên gạch hình chữ L che 3 ô còn lại.

$n = 2$: sàn 4×4

Ô trống ở góc trên bên trái (kí hiệu X):

Bước 1: X . . .

Đặt gạch trung tâm:

X . . .
. 1 1 .
. 1 . .
. . . .

Sau đó lát đệ quy từng góc. Kết quả cuối cùng:

X 2 3 3
2 2 1 4
5 1 1 4
5 5 4 4

Mỗi số là một viên gạch khác nhau.

19.6 Cài đặt minh họa (Python)

```
def tile(board, n, row, col, missing_row, missing_col, tile_id):
    if n == 1:
        return tile_id + 1

    half = n // 2
    center_row = row + half - 1
    center_col = col + half - 1

    # Xác định góc phần tư chứa ô trống
    missing_quadrant = 0
    if missing_row <= center_row:
        if missing_col <= center_col:
            missing_quadrant = 1 # top-left
        else:
            missing_quadrant = 2 # top-right
    else:
        if missing_col <= center_col:
            missing_quadrant = 3 # bottom-left
        else:
            missing_quadrant = 4 # bottom-right

    # Đặt gạch trung tâm
```



```

if missing_quadrant != 1:
    board[center_row][center_col] = tile_id
if missing_quadrant != 2:
    board[center_row][center_col + 1] = tile_id
if missing_quadrant != 3:
    board[center_row + 1][center_col] = tile_id
if missing_quadrant != 4:
    board[center_row + 1][center_col + 1] = tile_id

tile_id += 1

# Độ quy từng góc
# Góc trên bên trái
if missing_quadrant == 1:
    tile_id = tile(board, half, row, col, missing_row, missing_col, tile_id)
else:
    tile_id = tile(board, half, row, col, center_row, center_col, tile_id)

# Góc trên bên phải
if missing_quadrant == 2:
    tile_id = tile(board, half, row, col + half, missing_row, missing_col, tile_id)
else:
    tile_id = tile(board, half, row, col + half, center_row, center_col + 1, tile_id)

# Góc dưới bên trái
if missing_quadrant == 3:
    tile_id = tile(board, half, row + half, col, missing_row, missing_col, tile_id)
else:
    tile_id = tile(board, half, row + half, col, center_row + 1, center_col, tile_id)

# Góc dưới bên phải
if missing_quadrant == 4:
    tile_id = tile(board, half, row + half, col + half, missing_row, missing_col, tile_id)
else:
    tile_id = tile(board, half, row + half, col + half, center_row + 1, center_col + 1, tile_id)

return tile_id

# Khởi tạo
n = 4 # 2^n x 2^n, n=4 -> 16x16
size = 2 ** n
board = [[0] * size for _ in range(size)]
# Ô trống ở (0, 0)
tile(board, size, 0, 0, 0, 0, 1)

# In kết quả
for row in board:
    print(' '.join(f'{x:2d}' for x in row))

```

19.7 Kết luận

- Bài toán lát gạch hình chữ L là ứng dụng kinh điển của **chia để trị**.
- Mỗi bước chia sà thành 4 phần, đặt 1 viên gạch ở trung tâm để tạo "ô trống ảo" cho 3 phần còn lại.
- Số viên gạch cần dùng: $\frac{4^n - 1}{3}$.
- Độ phức tạp: $O(4^n)$ – tối ưu vì phải xử lý từng ô.